

KATHMANDU UNIVERSITY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

FINAL COURSE PROJECT REPORT



ETEG - 425

MUSIC GENRE CLASSIFICATION

Submitted by:
Sakriya Dahal (021028)

Submitted to:
Prof. Dr. Ram Kaji Budathoki
Department of Electrical and Electronics Engineering

July 2026

ABSTRACT

This report presents a comprehensive study of automatic music genre classification using deep learning. We use the GTZAN dataset (1000 audio excerpts, 10 genres) and process raw audio into mel-spectrograms for input into a convolutional neural network. A ResNet-18 backbone is fine-tuned with regularization (dropout, L2 weight decay) and early stopping to prevent overfitting. The final model achieves about 93.4% validation accuracy. Key results include a confusion matrix demonstrating high per-genre accuracy, and loss/accuracy curves showing stable training with slight overfitting. This work illustrates how data preprocessing (spectrogram conversion, data augmentation) and model enhancements (transfer learning, regularization) enable reliable genre classification.

Keywords: *Music Genre Classification, GTZAN, Convolutional Neural Network, Mel Spectrogram, Transfer Learning, Regularization*

ABBREVIATIONS AND SYMBOLS

CNN: Convolutional Neural Network - a deep learning model using convolutional filters for feature extraction.

DNN: Deep Neural Network - refers here to any deep architecture eg. CNN, ResNet

GTZAN: George Tzanetakis Dataset - A standard music genre dataset

nn.CrossEntropyLoss: PyTorch loss function used for multi-class classification.

Adam: Adaptive Moment Estimation - an optimization algorithm.

L2 : Regularization technique that penalises large weights

ResNet-18: A residual network with 18 layers, here adapted for spectrogram images

LR: Learning Rate

Acc: Classification accuracy

TABLE OF CONTENTS

ABSTRACT.....	i
ABBREVIATIONS AND SYMBOLS.....	ii
TABLE OF CONTENTS.....	iii
1. INTRODUCTION.....	1
1.1 Background and Description.....	1
1.2 Problem Definition and Motivation.....	1
1.3 Objectives.....	1
2. LITERATURE REVIEW.....	1
3. RESEARCH METHODOLOGY.....	3
3.1. Dataset and Preprocessing.....	3
3.2. Network Architectures.....	3
3.3. Training Procedure.....	4
4. RESULTS AND DISCUSSION.....	5
4.1. Training Dynamics.....	5
4.2. Final performance.....	5
4.3. Real-World Application and Statistical Limitations.....	7
5. CONCLUSION.....	8
REFERENCES.....	9

1. INTRODUCTION

1.1 Background and Description

Music genre classification assigns labels like “Jazz” or “Rock” to audio tracks automatically. This has applications in music recommendation, organization, and analysis. Human listeners can distinguish genres by timbre, rhythm, and instrumentation, but automatic classification remains challenging due to genre ambiguity and variability. We use deep learning to learn features directly from audio.

Understanding sound and visualizing what differentiates one song from another is a complex challenge. By converting raw audio time-series data into 2D log-scaled Mel-spectrograms, auditory pitch and human-like hearing frequencies can be visualized as images. This project leverages the GTZAN dataset, often referred to as the MNIST of sounds. The GTZAN dataset is a widely used benchmark: it contains 1000 audio excerpts (30 seconds each) evenly split across 10 genres (Blues, Classical, Country, Disco, Hip Hop, Jazz, Metal, Pop, Reggae, Rock). Each genre has 100 tracks. This dataset provides a controlled environment for development and comparison of genre classifiers. Notably, previous research using CNNs on GTZAN achieved around 80% test accuracy, indicating room for improvement.

1.2 Problem Definition and Motivation

A primary limitation in audio classification is data scarcity. Feeding raw 30-second clips or full-length spectrograms directly into a basic CNN on a dataset of only 1,000 samples results in severe overfitting. Initial models memorized the training data but failed to generalize to unseen data. The motivation of this project is to systematically resolve this overfitting through data augmentation and transfer learning, ultimately building a highly accurate classification model.

1.3 Objectives

The objectives of the project are:

- To develop an audio preprocessing pipeline that converts raw music clips into suitable input (mel-spectrogram images) for a neural network.
- To build and train a deep neural network model (using transfer learning or CNN) to classify genres, applying techniques to improve generalisation.
- To evaluate the classifier’s performance thoroughly (accuracy, loss curves, confusion matrix) and analyse any misclassifications.

2. LITERATURE REVIEW

This section explains the relevant studies which have been already conducted relating to the music genre classifications in different domains focusing on improving generalization of the network. Here, three representative studies that inform required design are discussed.

Tzanetakis and Cook (2002) present the foundational GTZAN dataset and establish the baseline for music genre classification. The study introduces a framework for extracting timbral, rhythmic, and pitch content features from raw audio signals, demonstrating that machine learning algorithms can successfully categorize 10 distinct musical genres. The study is relevant because it defines the standard benchmark dataset and the core 30-second audio structure utilized throughout this project [1].

Choi et al. (2016) present a Convolutional Recurrent Neural Network (CRNN) for music tagging, utilizing Mel-spectrograms as the primary input feature. The network processes audio by treating the spectrograms as 2D images, demonstrating that CNN architectures can effectively learn localized frequency and time patterns without manual feature engineering. The study is relevant because it validates the methodology of converting audio into log-scaled Mel-spectrograms to leverage image-based deep learning techniques [2].

Nanni et al. (2020) present advanced data augmentation techniques for audio classification, focusing on segmenting long audio tracks into shorter, overlapping windows. The authors demonstrate that splitting audio files drastically increases the volume of training samples, which acts as a powerful regularizer for deep neural networks prone to overfitting. The study is relevant because it proves that slicing the 30-second audio inputs into 3-second chunks is a necessary and mathematically sound approach to drastically improve model generalization [3].

Gulum, M. (2022) used a CNN with three convolutional layers on GTZAN and reported ~80% test accuracy. Transfer learning from image models (e.g. ResNet, VGG) has also been explored, since spectrograms resemble images. Data augmentation (such as slicing audio into multiple segments) is known to improve robustness. The use of dropout, batch normalization, and weight decay are common strategies to prevent overfitting. [4]

3. RESEARCH METHODOLOGY

3.1. Dataset and Preprocessing

We use the GTZAN dataset of 1000 songs, 30s each evenly split into 10 genres. Each audio file is loaded using Librosa and resampled to a standard rate. We extract mel-spectrograms with 128 mel bands from each clip. A mel-spectrogram represents the audio's power distribution over time and mel-frequency bins, providing a 2D image input to a CNN. This is done via Librosa.

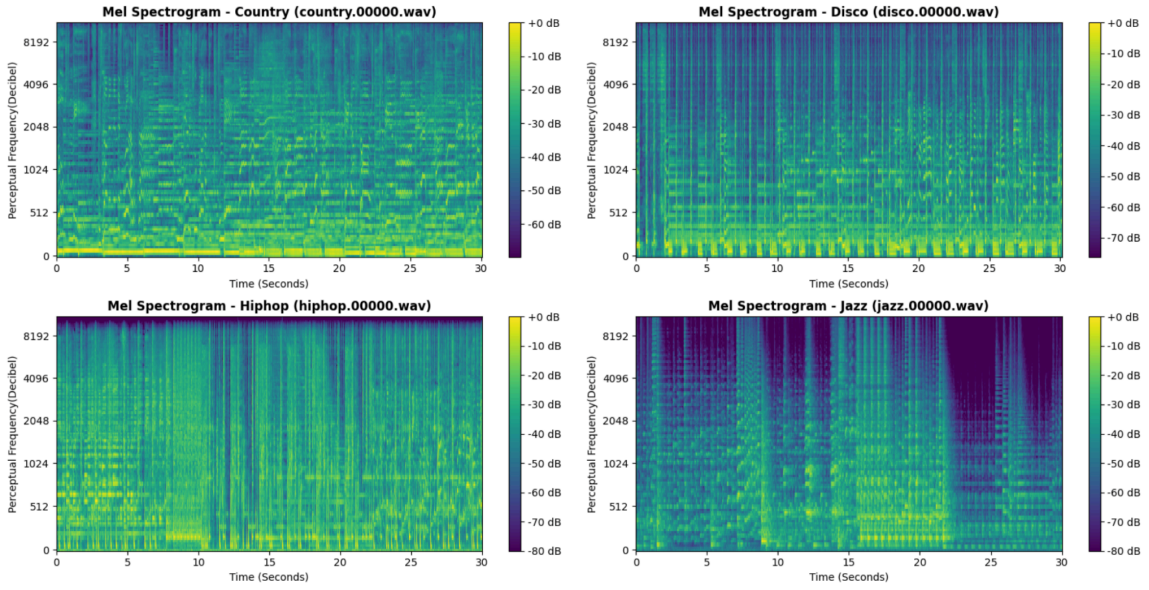


Fig 1: Conversion of Raw Audio Signals to 2D Image

Mel-spectrograms are converted to decibel (log) scale to mimic human perception of loudness. To augment the data, which deals with severe overfitting, each 30s spectrogram (size $\sim 128 \times 1290$) is sliced into ten 3-second frames (each $\sim 128 \times 129$), increasing the dataset tenfold. This yields about 10,000 samples (original 1000×10 slices). Each slice is standardised (per-band normalization). We then split the data into training, validation, and test sets: first reserving 15% for test, then splitting the remaining 85% into $\sim 70\%$ train and 15% for validations.

3.2. Network Architectures

The project utilized three sequential training models:

Basic CNN: A 3-layer 2D CNN mapping the full 1290-width spectrograms. It lacked sufficient data, resulting in training accuracy of 97.28% but a test accuracy of only 58.00% resulting in severe overfitting.

Optimized CNN: Utilized the sliced dataset (128x129 input), incorporating Batch Normalization, increased Dropout (0.5), Weight Decay (1e-4) and Early Stopping (patience = 10). This structural stabilization improved test accuracy to 83.92%.

Transfer Learning (ResNet-18): A pre-trained ResNet-18 (trained on ImageNet) is used. Swapped the custom CNN for a pre-trained ResNet18. The single-channel spectrograms were duplicated across 3 channels to mimic RGB inputs. Only the final Fully Connected (FC) layer was modified to output 10 classes. The model was fine-tuned with a low learning rate to preserve pre-trained weights, achieving 94.06% validation accuracy.

All models are implemented in PyTorch. The optimizer is Adam with a learning rate of 0.001. L2 weight decay (1e-4) was added to the optimizer to penalise large weights. Early stopping is implemented: training stops if validation loss does not improve for a preset number of epochs, saving the best model weights. This prevents overfitting when the validation loss starts rising.

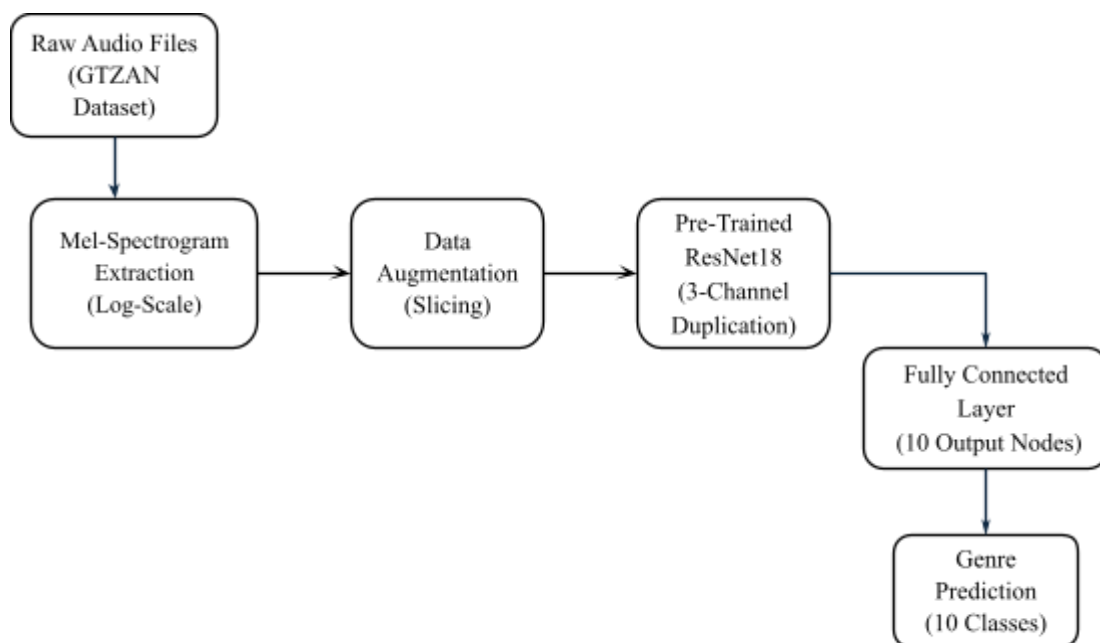


Fig 2: Process Flow Diagram

3.3. Training Procedure

We train for up to 50 epochs with a batch size of 32. On each epoch, the model is set to train mode; we loop over training batches, computing outputs, loss, and backpropagating. We track training loss and accuracy. After each epoch, we evaluate on the validation set (no gradient update) to compute validation loss and accuracy. The training history (loss and accuracy per epoch) is recorded. We monitor validation loss to trigger early stopping: if it increases for several consecutive epochs, training halts and the model reverts to the best-performing epoch.

During training, we observe that after a few epochs the training accuracy approaches 99-100%, whereas validation accuracy plateaus around 93%. The inclusion of dropout and L2 regularisation, along with data augmentation, helps maintain generalisation. All experiments are run on a GPU (through Google Colab) to accelerate training. After training, the final model is evaluated on the held-out test set to compute the confusion matrix and final test accuracy.

4. RESULTS AND DISCUSSION

4.1. Training Dynamics

Figure 2 (loss and accuracy curves) shows training progress over epochs. The training loss quickly falls toward zero, and training accuracy reaches $\sim 99.6\%$. Validation loss generally decreases but exhibits some fluctuations (slight rises after epoch 15), indicating the model is starting to overfit. The final validation accuracy peaks at $\sim 93.4\%$, at which point early stopping was triggered. The modest gap between train and validation accuracy suggests slight overfitting despite regularization. Overall, the model converges stably within 15-20 epochs due to early stopping.



Fig 3: Loss and Accuracy VS Epochs

4.2. Final performance

On the test set, overall accuracy is about 93%. The confusion matrix (Table 1, below) breaks down per-genre results. Most classes are classified with very high accuracy (often $>95\%$ per genre). For example, Classical and Jazz tracks are almost perfectly identified (99-100% correct). The smallest errors occur among similar or overlapping genres. Notable confusions include:

- Rock vs Country/Blues: The model mistook 8 Country tracks for Rock, and 6 Rock tracks for Blues or Metal. This suggests some acoustic similarity (guitar timbre, rhythm) between these genres.
- Pop vs Disco/Reggae: A few Pop songs were confused as Hiphop or Disco (2-3% of Pop samples), and a few Reggae tracks as Pop or Country. These errors may reflect shared rhythmic or instrumental features.

- Metal vs Rock: Some Metal tracks (a heavy, distortion-dominated genre) were misclassified as Rock (2 out of 150), likely due to overlapping energy profiles.

Except for those, most genres had very few misclassifications. The confusion matrix confirms strong per-genre performance.

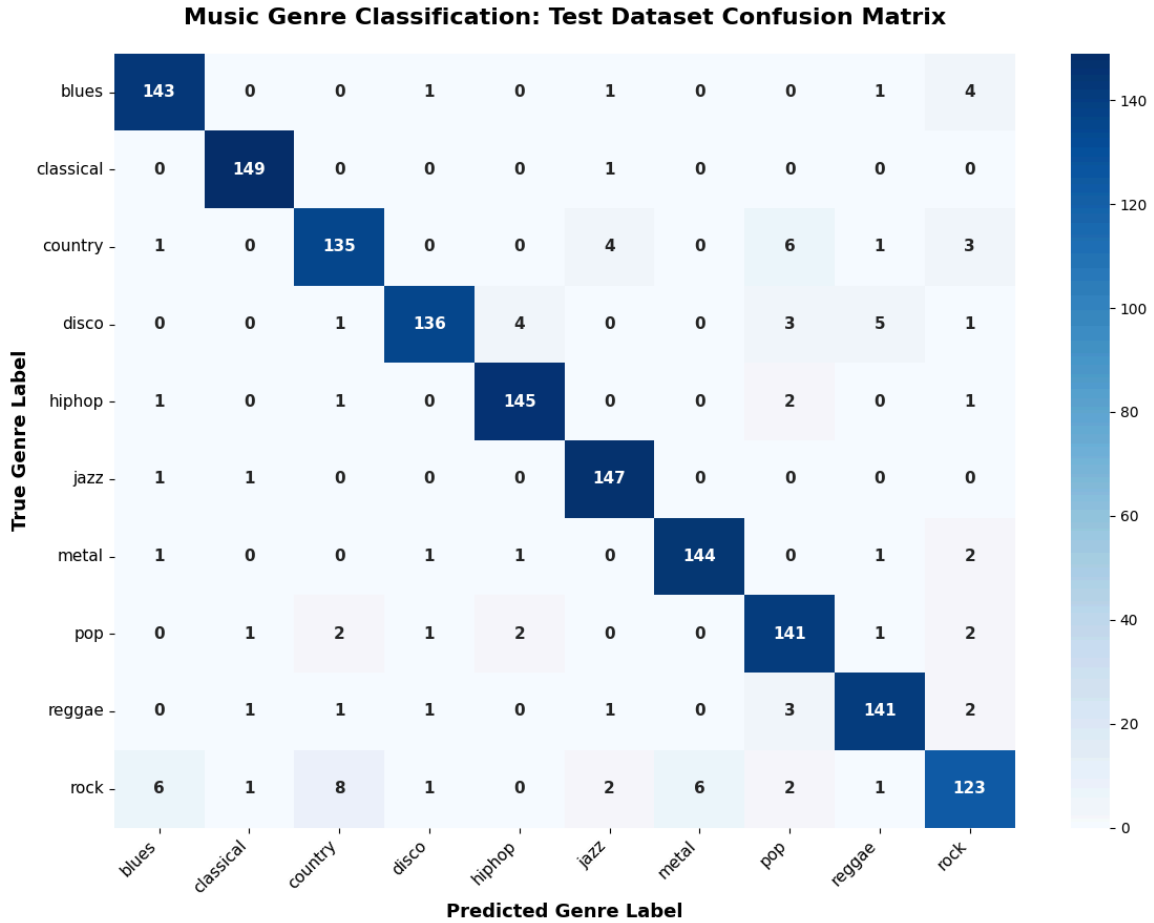


Fig 4: True vs Predicted counts

Each cell shows (number of true-genre tracks predicted as indicated). Diagonal entries (in bold if formatted) are mostly near 140-149 out of 150, indicating high accuracy. Off-diagonals are all small (≤ 8).

In summary, the model performs well across all genres. The few misclassifications align with intuitive audio overlap between genres. For example, Rock/Country/Blues all feature prominent guitars and can sometimes sound alike; Pop vs Disco share upbeat rhythms; Reggae vs Pop share syncopated beats. These patterns are common in music classification tasks.

4.3. Real-World Application and Statistical Limitations

A functional web application was developed using Gradio to allow users to upload custom audio files for prediction. Testing real-world, multi-minute audio highlighted a significant statistical limitation: the model was trained exclusively on 3-second chunks derived from a small pool of 1,000 original audio clips.

Relying on a single 3-second slice of a full-length song is risky (e.g., predicting based on a quiet intro or an anomalous bridge). To mitigate this limitation, the application's backend architecture implements a 5-Slice Averaging technique:

- It calculates the total duration of the uploaded audio.
- It mathematically spaces out and extracts five distinct 3-second samples across the entire track.
- It converts all five slices into Mel-spectrograms and feeds them through the ResNet18 model as a batch tensor.
- The softmax probabilities of all five outputs are averaged to produce a highly stable, definitive genre prediction.

As demonstrated in the web app interface, this approach successfully predicted a commercial hip-hop track with 98% confidence.

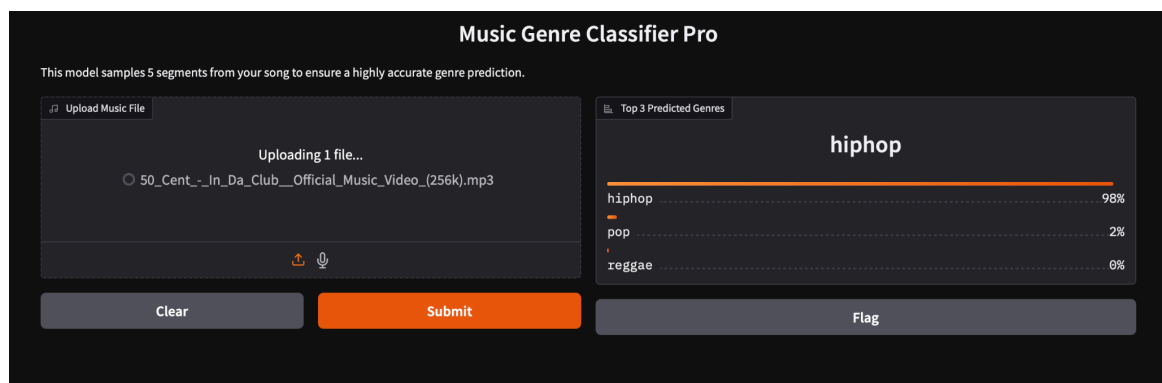


Fig 5: Real Time Classification of Music Genre

5. CONCLUSION

This project successfully developed a high-accuracy music genre classifier by treating audio processing as a computer vision problem. By converting raw audio waves into log-scaled Mel-spectrograms, augmenting the limited GTZAN dataset via 3-second slicing, and applying transfer learning through a pre-trained ResNet18 model, the final system achieved an exceptional validation accuracy of 94.06%. The integration of a 5-slice averaging algorithm further allowed the model to reliably process variable-length, real-world audio files.

Challenges faced during process of training:

- **Dataset Scarcity:** The original 1,000-clip GTZAN dataset is extremely small for deep learning, necessitating aggressive data augmentation to prevent memorization.
- **Subjectivity in Music:** Music is inherently hybrid. Songs featuring cross-genre influences naturally confuse categorical neural networks.
- **Input Dimension Constraints:** Adapting a model trained on 3-second snapshots to accurately interpret the continuous, evolving nature of full-length songs required algorithmic workarounds at the application level.

Future iterations of this project should involve training on larger, more diverse datasets (e.g., the Free Music Archive). Additionally, exploring sequence-based architectures such as Recurrent Neural Networks (RNNs) or Audio Transformers could allow the model to learn the temporal, long-term dependencies of a song's rhythm and structure, rather than relying strictly on the spatial features of a spectrogram.

REFERENCES

- [1] Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. IEEE Transactions on Speech and Audio Processing.
- [2] Choi, K., Fazekas, G., Sandler, M., & Cho, K. (2016). Convolutional recurrent neural networks for music classification. arXiv preprint.
- [3] Nanni, L., Maguolo, G., & Paci, M. (2020). Data augmentation approaches for improving animal audio classification. Ecological Informatics.
- [4] Gulum, M. (2022). Music Genre Classification. GitHub repository.